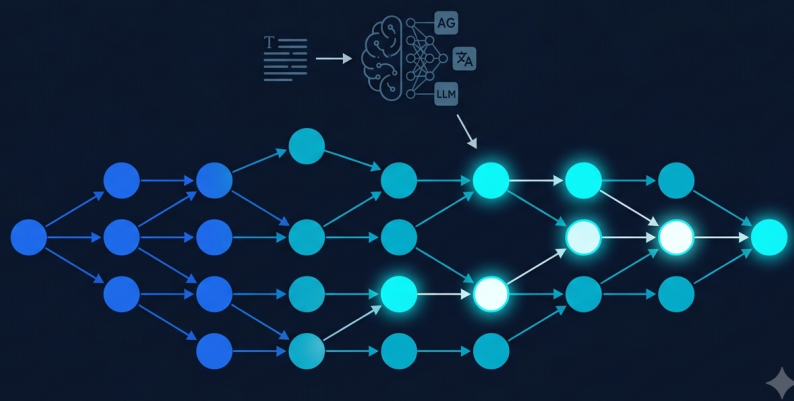




TEMA 3

Planificación de Trayectoria Profesional con LLM

Fabio Víctor Alonso Bañobre

Grupo C-311

Facultad de Matemática y Computación (MATCOM)
Universidad de La Habana

Descripción del Problema

Contexto y motivación

En el ámbito de la formación tecnológica, una persona que desea alcanzar un perfil profesional concreto —como *Data Scientist*, *ML Engineer* o *Backend Developer*— se enfrenta a un catálogo extenso de cursos con dependencias entre sí: no es posible estudiar *Machine Learning* sin antes dominar álgebra lineal y programación, del mismo modo que no tiene sentido tomar un curso avanzado de bases de datos antes de conocer SQL básico.

Determinar manualmente qué cursos tomar, en qué orden y cuáles omitir es una tarea combinatoriamente costosa que además requiere conocimiento experto del dominio. El sistema **Career Path Planner** aborda este problema de forma automática, combinando algoritmos de búsqueda clásicos con la capacidad semántica de un modelo de lenguaje grande (LLM).

Definición del problema

Dado:

- Un **catálogo de cursos**, donde cada curso enseña un conjunto de habilidades y requiere que ciertas habilidades ya estén adquiridas (prerrequisitos).
- Un **perfil profesional objetivo**, expresado bien como un conjunto estructurado de habilidades requeridas, bien como una descripción en lenguaje natural (“*quiero trabajar como ML Engineer*”).
- Un **estado inicial** del usuario, descrito por las habilidades que ya posee.

Se busca encontrar una **secuencia ordenada y válida de cursos** que:

1. Respete todas las restricciones de prerrequisitos (ningún curso se toma antes de que sus precondiciones estén satisfechas).
2. Permita al usuario alcanzar todas las habilidades requeridas por el perfil objetivo.
3. Sea **óptima** respecto a un criterio de costo: mínimo número de cursos, mínima duración total en semanas, o una combinación de ambos.

Dificultades inherentes al problema

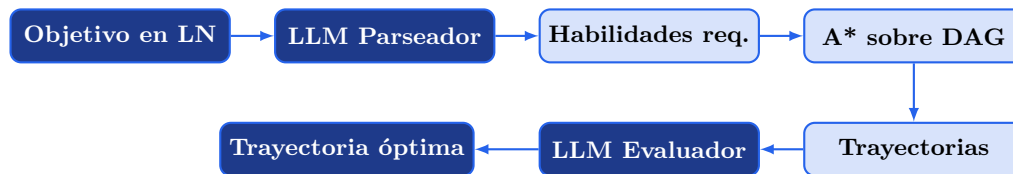
El problema presenta tres fuentes de complejidad que justifican el diseño del sistema:

1. **Explosión combinatoria.** El espacio de secuencias posibles crece exponencialmente con el número de cursos. Con un catálogo de n cursos, el número de ordenaciones posibles es $O(n!)$, lo que hace inviable la búsqueda exhaustiva para catálogos medianos o grandes.

2. **Restricciones de precedencia.** Las dependencias entre cursos forman una red de restricciones que cualquier solución válida debe satisfacer. No basta con encontrar un camino; hay que encontrar uno que respete el orden parcial impuesto por los prerrequisitos.
3. **Entrada en lenguaje natural.** El usuario no expresa su objetivo como un conjunto formal de habilidades, sino con frases como “*quiero trabajar en inteligencia artificial*”. Mapear esta intención al espacio formal de habilidades del catálogo requiere comprensión semántica que los algoritmos clásicos no poseen.

Estas tres dificultades motivan una arquitectura híbrida: el LLM resuelve la interpretación semántica (dificultad 3) y el algoritmo A* resuelve la búsqueda óptima bajo restricciones (dificultades 1 y 2).

El diagrama siguiente resume el flujo completo del sistema:



Modelado Formal

Representación como problema de búsqueda en espacio de estados

El problema se modela como un **problema de búsqueda** sobre un grafo dirigido acíclico (DAG), donde cada nodo representa un estado de conocimiento del usuario y cada arco representa la acción de completar un curso.

Definición 1 (Problema de búsqueda). Se define formalmente la tupla:

$$\mathcal{P} = \langle \mathcal{S}, s_0, \mathcal{A}, T, c, G \rangle$$

donde cada componente se define como sigue:

Componente	Definición
$\mathcal{H}$	Conjunto finito de todas las habilidades del catálogo.
$\mathcal{S}$	Espacio de estados: $\mathcal{S} = 2^{\mathcal{H}}$ (todos los subconjuntos posibles de habilidades).
s_0	Estado inicial: $s_0 = H_{\text{ini}} \subseteq \mathcal{H}$, habilidades que el usuario ya posee.
$\mathcal{C}$	Catálogo de cursos. Cada curso c_i es la tupla $\langle \text{pre}_i, \text{hab}_i, d_i, \ell_i \rangle$ donde $\text{pre}_i \subseteq \mathcal{H}$ son los prerrequisitos, $\text{hab}_i \subseteq \mathcal{H}$ las habilidades enseñadas, $d_i \in \mathbb{Z}^+$ la duración en semanas y $\ell_i \in \{\text{prin.}, \text{inter.}, \text{avanz.}\}$ el nivel.
$\mathcal{A}(s)$	Acciones aplicables en estado s : un curso c_i es aplicable si $\text{pre}_i \subseteq s$ y $\text{hab}_i \not\subseteq s$ (aporta habilidades nuevas).
T	Función de transición: $T(s, c_i) = s \cup \text{hab}_i$.
c	Función de costo de una acción. Según el criterio elegido: $c_{\text{cursos}}(s, c_i) = 1$ $c_{\text{semanas}}(s, c_i) = d_i$ $c_{\text{balance}}(s, c_i) = \alpha \cdot \frac{d_i}{10} + (1 - \alpha)$
G	Objetivo: $G = \{s \in \mathcal{S} \mid H^* \subseteq s\}$, donde $H^* \subseteq \mathcal{H}$ son las habilidades requeridas por el perfil profesional.

Observación. El estado s no registra explícitamente los cursos tomados sino las habilidades adquiridas. Esto comprime el espacio de estados y elimina estados redundantes: dos secuencias de cursos distintas que producen el mismo conjunto de habilidades corresponden al mismo nodo en el grafo de búsqueda.

Modelado como DAG de cursos

El catálogo de cursos define un **grafo dirigido acíclico** sobre habilidades. Formalmente:

Definición 2 (DAG de prerrequisitos). Sea $\mathcal{H}$ el conjunto de habilidades. Se define el grafo $G_{\text{DAG}} = (\mathcal{H}, E)$ donde existe una arista $h_j \rightarrow h_k$ si existe algún curso c_i tal que $h_j \in \text{pre}_i$ y $h_k \in \text{hab}_i$. El sistema garantiza que G_{DAG} es acíclico, propiedad verificada automáticamente por `validate_dataset.py`.

La aciclicidad del grafo es una condición necesaria para la *completitud* del algoritmo de búsqueda: si existiera un ciclo de dependencias, ninguna secuencia de cursos podría satisfacer todos sus prerrequisitos simultáneamente.

Modelado como problema CSP

Las restricciones de prerequisites se modelan como un **Problema de Satisfacción de Restricciones** (CSP): $\langle X, D, R \rangle$, donde:

- $X = \{x_1, x_2, \dots, x_n\}$ son variables de posición, con x_k representando el k -ésimo curso en la secuencia.
- $D_k = \mathcal{C}$ para todo k (cada posición puede asignarse cualquier curso del catálogo).
- R es el conjunto de restricciones binarias: para todo curso c_i en posición k , cada prerequisite $h \in \text{pre}_i$ debe ser enseñado por algún curso en una posición anterior $j < k$.

El sistema implementa la verificación de estas restricciones mediante la función `validar_trayectoria`, que recorre la secuencia y comprueba que cada curso puede tomarse dado el estado de habilidades acumulado hasta ese punto.

Modelado STRIPS de los cursos

Cada curso c_i del catálogo se representa como una **acción STRIPS**:

$$c_i = \langle \text{Pre}(c_i), \text{Add}(c_i), \text{Del}(c_i) \rangle$$

con:

- $\text{Pre}(c_i) = \text{pre}_i$: habilidades que deben estar en el estado antes de tomar el curso.
- $\text{Add}(c_i) = \text{hab}_i$: habilidades que se incorporan al estado al completarlo.
- $\text{Del}(c_i) = \emptyset$: ninguna habilidad se pierde (la adquisición de conocimiento es monótonica).

El estado objetivo es $\text{Goal} = H^*$, el conjunto de habilidades del perfil profesional deseado. Resolver el problema STRIPS equivale a encontrar la secuencia de cursos que transforma s_0 en un estado que satisface Goal , que es exactamente lo que resuelve A^* sobre el DAG.

Heurística admisible para A^*

Definición 3 (Heurística h). Para un estado s y un perfil objetivo con habilidades requeridas H^* :

$$h(s) = |H^* \setminus s|$$

es decir, el número de habilidades del objetivo aún no adquiridas.

Admisibilidad. Cada habilidad faltante requiere al menos un curso para ser adquirida, por lo que $h(s)$ es cota inferior del costo real $h^*(s)$. Bajo el criterio de costo unitario por curso se tiene directamente $h(s) \leq h^*(s)$. Bajo el criterio de semanas, la cota se mantiene porque cada curso dura al menos 1 semana y aporta al menos una habilidad nueva. La admisibilidad garantiza que A^* siempre encuentra la trayectoria de costo mínimo cuando existe solución.

Resumen del modelo

La tabla siguiente relaciona los contenidos teóricos de la asignatura con los componentes implementados en el sistema:

Técnica	Rol en el sistema
Búsqueda A*	Algoritmo principal. Encuentra la trayectoria óptima sobre el DAG respetando prerequisites.
Greedy Best-First	Variante de comparación. Más rápida pero sin garantía de optimalidad.
CSP	Modelado de prerequisites como restricciones binarias. El validador actúa como verificador de consistencia.
Planificación STRIPS	Cada curso es una acción Pre/Add/Del. El objetivo es el perfil profesional.
PLN / LLM	Parseador: transforma el objetivo en lenguaje natural en el conjunto formal H^* .
Sistemas expertos	Evaluador LLM: puntúa coherencia y calidad de la trayectoria generada.
Simulación Monte Carlo	Evaluación estadística bajo incertidumbre en duraciones, usando distribución triangular (TTI).

Dataset

Descripción general

El dataset utilizado en el sistema es un catálogo sintético de cursos tecnológicos diseñado específicamente para este proyecto. Recibe el nombre `dataset.json` y constituye la fuente única de conocimiento sobre la que operan los algoritmos de búsqueda.

El dataset contiene **150 cursos**, **141 habilidades** y **11 perfiles profesionales**, organizados en un grafo dirigido acíclico (DAG) de dependencias verificado automáticamente. La tabla siguiente resume sus dimensiones principales:

Elemento	Cantidad
Cursos totales	150
Habilidades	141
Perfiles profesionales	11
Cursos sin prerequisites	3
Prerequisites promedio por curso	3,0
Duración promedio (semanas)	6,1
Duración mínima / máxima	2 / 12 semanas

Estructura del dataset

Cada entrada del catálogo sigue el esquema siguiente:

Campo	Tipo	Descripción
<code>id</code>	string	Identificador único (c01–c150).
<code>nombre</code>	string	Nombre legible del curso.
<code>descripcion</code>	string	Resumen del contenido cubierto.
<code>prerrequisitos</code>	lista	Habilidades que el usuario debe poseer antes de tomarlo.
<code>habilidades</code>	lista	Habilidades que el curso enseña al completarlo.
<code>duracion_semanas</code>	entero	Duración estimada en semanas (rango: 2–12).
<code>nivel</code>	string	Uno de: <code>principiante</code> , <code>intermedio</code> , <code>avanzado</code> .

Los perfiles profesionales siguen un esquema análogo, con campos `id`, `nombre`, `descripcion` y `habilidades_requeridas`: el conjunto H^* que define el estado objetivo en el problema de búsqueda.

Distribución por nivel y prerequisites

Los 150 cursos se distribuyen equilibradamente entre los tres niveles académicos, garantizando que el sistema pueda planificar trayectorias completas para usuarios con cualquier punto de partida:

Nivel	Cursos	Porcentaje	Duración media (sem.)
Principiante	44	29,3 %	3,2
Intermedio	54	36,0 %	5,9
Avanzado	52	34,7 %	9,8
Total	150	100 %	6,1

En cuanto a la estructura de dependencias, la distribución de prerrequisitos por curso es la siguiente:

Prerrequisitos	Cursos	Porcentaje
Sin prerrequisitos (nodos raíz)	3	2,0 %
1-2 prerrequisitos	61	40,7 %
3-4 prerrequisitos	59	39,3 %
5 o más prerrequisitos	27	18,0 %

Los tres cursos raíz (sin prerrequisitos) son *Lógica y Programación*, *Matemáticas para Tecnología* e *HTML y CSS*, que representan los puntos de entrada naturales al grafo para usuarios sin conocimientos previos.

Perfiles profesionales

El dataset define 11 perfiles profesionales del área tecnológica, cada uno con entre 6 y 10 habilidades requeridas. Los tres perfiles que sirven de base para los experimentos del proyecto son:

Perfil	Habilidades req.	Descripción
Data Scientist	8	Análisis de datos, estadística, ML y visualización.
Backend Developer	7	APIs, bases de datos, Docker y microservicios.
ML Engineer	7	ML, deep learning, despliegue en producción con MLOps.

El catálogo completo incluye además: *Frontend Developer*, *Full Stack Developer*, *DevOps Engineer*, *Data Engineer*, *Cloud Architect*, *Cybersecurity Analyst*, *AI Researcher* y *Mobile Developer*.

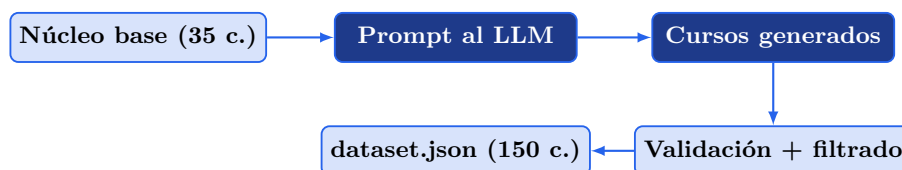
Proceso de generación

El dataset se construyó en dos fases complementarias:

Fase 1 — Núcleo base (c01–c35). Se diseñaron manualmente 35 cursos que representan el tronco curricular fundamental del área tecnológica: fundamentos de programación, matemáticas, bases de datos, frameworks web, contenedores y algoritmos de ML/DL. Este núcleo establece las dependencias educativas canónicas del dominio (por ejemplo, *Estadística Básica* → *Estadística Inferencial* → *Machine Learning*).

Fase 2 — Expansión con LLM (c36–c150). El script `generate_synthetic_dataset.py` llamó a la API de Hugging Face (modelo `Qwen/Qwen2.5-72B-Instruct`) en lotes de 5 cursos, indicando en cada petición el catálogo de habilidades ya existente, el nivel académico del lote y el dominio tecnológico objetivo. El LLM generó los nuevos cursos en formato JSON estricto. Cada respuesta fue validada y normalizada antes de incorporarse al dataset: prerequisites no reconocidos se descartaron, duraciones fuera del rango permitido se ajustaron y nombres duplicados se rechazaron. De este modo se generaron 115 cursos adicionales, triplicando el catálogo base con una razón de expansión de 3,3×.

El proceso completo, incluyendo la generación de perfiles profesionales, se resume en el siguiente esquema:



Justificación de validez

Para que el dataset sea válido como entrada al sistema, debe cumplir tres condiciones formales. Se argumenta a continuación que el proceso de generación las garantiza:

- 1. Aciclicidad del grafo (DAG válido).** La herramienta `fix_dag.py` detecta y elimina prerequisites circulares mediante el algoritmo de Kahn aplicado al grafo de dependencias entre cursos. Posteriormente, `validate_dataset.py` verifica la ausencia de ciclos antes de usar el dataset. En el dataset final no existen ciclos.
- 2. Coherencia educativa de las dependencias.** El núcleo base (Fase 1) fue diseñado respetando progresiones curriculares reales: primero lógica y matemáticas, luego estadística e inferencia, luego *machine learning* y finalmente *deep learning* y MLOps. El LLM recibió en cada prompt el catálogo de habilidades existente y tenía prohibido crear prerequisites fuera de él, por lo que los cursos generados solo amplían el grafo sin alterar la progresión establecida.
- 3. Alcanzabilidad de todos los perfiles.** `validate_dataset.py` verifica que cada habilidad requerida por cualquier perfil profesional es enseñada por al menos un curso del catálogo. Esta

propiedad garantiza que el sistema siempre puede encontrar solución cuando el perfil es alcanzable desde el estado inicial dado.

Diseño del Algoritmo

Visión general

El sistema implementa tres algoritmos de búsqueda sobre el DAG de cursos, cada uno con un rol distinto dentro del diseño experimental:

Algoritmo	Rol	Garantía
A* (criterio cursos)	Principal	Óptimo: mínimo número de cursos.
A* (criterio semanas)	Principal	Óptimo: mínima duración total.
A* (criterio balance)	Principal	Óptimo: combinación ponderada.
Greedy Best-First	Comparación	Sin garantía de optimalidad.
K-mejores	Extensión	Genera k trayectorias distintas.

Los tres criterios de A* permiten al sistema responder a objetivos distintos del usuario: minimizar el esfuerzo (número de cursos), minimizar el tiempo (semanas) u optimizar ambos simultáneamente.

Representación del estado

El estado de búsqueda se representa como un **conjunto inmutable de habilidades adquiridas** (`frozenset`). Esta elección de representación tiene tres consecuencias importantes:

1. **Compresión del espacio de estados.** Dos secuencias de cursos distintas que producen el mismo conjunto de habilidades son el mismo estado. El algoritmo no reexpande estados ya visitados con mejor costo.
2. **Hashabilidad.** Los `frozenset` son hashables en Python, lo que permite usarlos como claves del diccionario `g_score` (costo acumulado) y de `came_from` (reconstrucción del camino) en tiempo $O(1)$.
3. **Monotonidad.** La función de transición $T(s, c_i) = s \cup \text{hab}_i$ garantiza que el estado nunca decrece: una habilidad adquirida no se puede perder. Esto hace que el espacio de búsqueda sea finito y acíclico.

Los cursos ya tomados se deducen del estado actual en lugar de almacenarse explícitamente: un curso c_i se considera tomado si $\text{hab}_i \subseteq s$, lo que evita estados redundantes.

Algoritmo A*

A* mantiene una cola de prioridad (*min-heap*) ordenada por la función de evaluación $f(s) = g(s) + h(s)$, donde $g(s)$ es el costo acumulado desde el estado inicial y $h(s)$ es la heurística admisible definida en la Sección 2.

El pseudocódigo del algoritmo implementado es el siguiente:

Algorithm 1 A* sobre el DAG de cursos

Require: Grafo G , estado inicial s_0 , perfil objetivo p , criterio c

Ensure: Trayectoria óptima o fallo

```
1: frontera  $\leftarrow$  MinHeap( $\{(h(s_0), 0, s_0)\}$ )
2:  $g[s_0] \leftarrow 0$ ;  $\text{came\_from} \leftarrow \{\}$ 
3: while frontera  $\neq \emptyset$  do
4:    $(f, \_, s) \leftarrow \text{frontera.pop}()$ 
5:   if nodo obsoleto then
6:     continue
7:   end if
8:   if  $G.\text{es\_objetivo}(s, p)$  then
9:     return reconstruir( $\text{came\_from}, s$ )
10:  end if
11:  tomados  $\leftarrow \{c_i \mid \text{hab}_i \subseteq s\}$ 
12:  for  $c_i \in G.\text{cursos\_disponibles}(s, \text{tomados})$  do
13:     $s' \leftarrow s \cup \text{hab}_i$ 
14:     $g' \leftarrow g[s] + \text{costo}(c_i, c)$ 
15:    if  $g' < g[s']$  then
16:       $g[s'] \leftarrow g'$ ;  $\text{came\_from}[s'] \leftarrow (c_i, s)$ 
17:      frontera.push( $(g' + h(s'), \text{tie}, s')$ )
18:    end if
19:  end for
20: end while
21: return sin solución
```

Función de costo. Según el criterio elegido:

$$\text{costo}(c_i, \text{cursos}) = 1 \quad \text{costo}(c_i, \text{semanas}) = d_i \quad \text{costo}(c_i, \text{balance}) = \alpha \cdot \frac{d_i}{10} + (1 - \alpha)$$

donde $\alpha = 0,5$ por defecto, dando igual peso a duración y cantidad de cursos.

Desempate determinista. Cuando dos nodos tienen el mismo valor f , se desempata por un contador incremental (**tiebreak**) que preserva el orden FIFO de inserción, asegurando que el algoritmo sea **determinista** y reproducible ante el mismo estado inicial.

Límite de expansión. A* acepta un parámetro **max_nodos** (por defecto 500 000) que detiene la búsqueda si el grafo es demasiado grande, evitando consumo de memoria no acotado. En los experimentos con el dataset de 150 cursos este límite nunca se alcanzó.

Optimalidad. Dado que h es admisible (demostrado en la Sección 2) y los costos son no negativos, A* garantiza encontrar la trayectoria de costo mínimo cuando existe solución.

Greedy Best-First Search

El algoritmo Greedy expande siempre el estado con menor valor heurístico $h(s)$, sin considerar el costo acumulado $g(s)$. Su implementación usa un conjunto de *visitados* en lugar del diccionario `g_score`, lo que lo hace más rápido pero sin garantía de optimalidad.

Las diferencias clave respecto a A* son:

Aspecto	A*	Greedy
Función de prioridad	$f = g + h$	$f = h$
Revisita de estados	Sí (si hay mejor g)	No
Garantía de óptimo	Sí	No
Velocidad típica	Más lenta	Más rápida
Nodos expandidos	Más	Menos

Greedy se incluye como **línea base experimental**: su comparación con A* cuantifica el precio que se paga en calidad de solución a cambio de eficiencia computacional.

Generación de K mejores trayectorias

El sistema implementa una variante de *Yen's k-shortest paths* adaptada al dominio de cursos. El algoritmo genera hasta k trayectorias distintas de forma iterativa:

1. Se obtiene la primera trayectoria con A* estándar.
2. En cada iteración i , se seleccionan como candidatos a bloquear los cursos más costosos de la trayectoria anterior (ordenados por $\text{costo}(c_j, \text{criterio})$ descendente).
3. Para cada candidato c_j , se construye un grafo restringido $G' = G \setminus \{c_j\}$ y se ejecuta A* con un límite de nodos reducido (50 000).
4. Se acepta la alternativa con menor costo que no sea idéntica a ninguna solución ya encontrada.
5. El curso bloqueado en la iteración ganadora se añade al conjunto de bloqueos globales para evitar ciclos.

Esta estrategia produce trayectorias genuinamente distintas porque cada una evita al menos un curso de la anterior, ofreciendo al usuario opciones reales de personalización.

Validación de trayectorias

Toda trayectoria generada por cualquiera de los algoritmos se verifica con la función `validar_trayectoria`, que comprueba dos condiciones de forma secuencial:

1. **Validez de prerequisites.** Para cada curso c_i en posición k , se verifica que $\text{pre}_i \subseteq s_{k-1}$, donde s_{k-1} es el estado acumulado tras los $k - 1$ cursos anteriores. Si la condición falla, se reporta qué prerequisite no estaba satisfecho y en qué posición.
2. **Alcance del objetivo.** Tras recorrer toda la secuencia, se verifica que $H^* \subseteq s_{\text{final}}$.

En todos los experimentos realizados, el 100 % de las trayectorias generadas por A* y Greedy pasaron la validación, confirmando la correctitud de la implementación.

Complejidad computacional

Algoritmo	Tiempo (peor caso)	Espacio
A*	$O(b^d \log b^d)$	$O(b^d)$
Greedy	$O(b^d \log b^d)$	$O(b^d)$

Donde b es el factor de ramificación (cursos disponibles en cada estado) y d es la profundidad de la solución (número de cursos en la trayectoria). En la práctica, la heurística admisible reduce drásticamente los nodos expandidos por A* respecto al peor caso teórico, como se cuantifica en **Resultados y Análisis**.

Rol del LLM en el Sistema

Justificación de la integración

Los algoritmos de búsqueda clásicos operan sobre representaciones formales: estados, acciones y costos bien definidos. Sin embargo, el problema de planificación de trayectoria profesional presenta dos componentes que escapan a ese formalismo:

1. El usuario expresa su objetivo en **lenguaje natural**, no como un conjunto de habilidades del catálogo.
2. La calidad de una trayectoria no solo depende de su costo numérico, sino de su **coherencia semántica**: si los cursos son realmente relevantes para el objetivo, si el orden tiene sentido pedagógico, si hay redundancias o lagunas.

El LLM resuelve ambos problemas actuando en dos roles completamente diferenciados dentro del *pipeline*. No es un componente decorativo: sin el parseador, el sistema no puede recibir entrada en lenguaje natural; sin el evaluador, no puede juzgar la calidad semántica de la solución.

Arquitectura de integración

El sistema usa la API de Hugging Face Inference con el modelo `Qwen/Qwen2.5-72B-Instruct` como primera opción, con fallback automático a `Mistral-7B-Instruct-v0.3` y `zephyr-7b-beta` si el modelo principal falla por rate limit o indisponibilidad. La clave de API se gestiona mediante variables de entorno (`.env`), nunca hardcodeada en el código.

Ambos roles del LLM comparten la misma infraestructura de llamada, que incluye reintentos automáticos con espera adaptativa ante errores 429 (rate limit) y 503 (modelo cargando), y un parser JSON robusto que maneja respuestas con markdown, comillas tipográficas o texto extra alrededor del JSON esperado.

Rol 1 — Parseador de objetivos

Función: transformar una descripción en lenguaje natural del objetivo profesional del usuario en un conjunto estructurado de habilidades del catálogo.

Entrada / Salida:

“quiero trabajar como ML Engineer” $\xrightarrow{\text{LLM}}$ `{python_avanzado, machine_learning, docker, ...}`

Diseño del prompt. El prompt incluye cuatro elementos clave que guían la respuesta del modelo:

1. El **objetivo del usuario** entre comillas.
2. Las **habilidades ya adquiridas**, para que el modelo no sugiera habilidades que el usuario ya posee.
3. El **catálogo completo** de habilidades disponibles, para que solo seleccione habilidades que existen en el sistema.
4. Un **ejemplo few-shot** que muestra el formato JSON esperado y el nivel de especificidad requerido.

El modelo responde con un objeto JSON con los campos: `habilidades_requeridas`, `perfil_detectado`, `confianza` y `razon`. La respuesta se filtra automáticamente contra el catálogo: las habilidades no reconocidas se descartan, las ya adquiridas se eliminan, y solo las habilidades válidas y nuevas se pasan al algoritmo de búsqueda como H^* .

Manejo de alucinaciones. El LLM puede generar habilidades que no existen en el catálogo (por ejemplo, `pytorch_avanzado` en lugar de `deep_learning`). El sistema las detecta, las registra en el campo `habilidades_invalidas` del resultado y las descarta sin interrumpir el flujo. Si tras el filtrado quedan al menos una habilidad válida, la búsqueda procede normalmente.

Mapeo al perfil. Las habilidades válidas extraídas se usan para seleccionar el perfil del catálogo con mayor solapamiento, mediante la función:

$$p^* = \arg \max_{p \in \mathcal{P}} |H_{\text{LLM}} \cap H_p^*|$$

Este perfil p^* es el que recibe el algoritmo A^* como estado objetivo.

Rol 2 — Evaluador de trayectorias

Función: evaluar semánticamente la trayectoria generada por A^* y producir una puntuación de calidad con justificación narrativa.

Entrada / Salida:

(objetivo, trayectoria) $\xrightarrow{\text{LLM}}$ {puntuacion : 8,5, fortalezas : [...], debilidades : [...], sugerencias : [...]}

Diseño del prompt. El evaluador recibe en el prompt:

1. El **objetivo profesional** y el **perfil objetivo**.
2. Las **habilidades iniciales** del usuario, para contextualizar si la trayectoria es adecuada para su punto de partida.
3. La **trayectoria completa** con nombre, duración y nivel de cada curso, y las habilidades que cada uno enseña.
4. **Criterios de evaluación explícitos** que guían la respuesta del modelo de forma determinista.

Los criterios de evaluación instruyen al modelo a: *penalizar trayectorias con más de 12 cursos o más de 50 semanas* (puntuación máxima 7), *recompensar trayectorias concisas de 5–7 cursos y menos de 35 semanas* (puntuación 8–10), evaluar la coherencia pedagógica de la secuencia (de menor a mayor nivel) y detectar cursos redundantes o irrelevantes para el objetivo.

El modelo responde con un JSON con los campos: **puntuacion** (0–10), **nivel_calidad** (*excelente* / *bueno* / *aceptable* / *deficiente*), **fortalezas**, **debilidades**, **sugerencias** y **resumen**.

Valor semántico del evaluador. A^* garantiza optimalidad formal (mínimo costo numérico), pero no puede detectar si una trayectoria incluye cursos poco relevantes para el objetivo real o si la secuencia es pedagógicamente incoherente. El LLM cubre exactamente esta brecha: actúa como un experto en carreras tecnológicas que valida la solución desde una perspectiva cualitativa que el algoritmo no puede proporcionar.

Evaluador simulado (fallback)

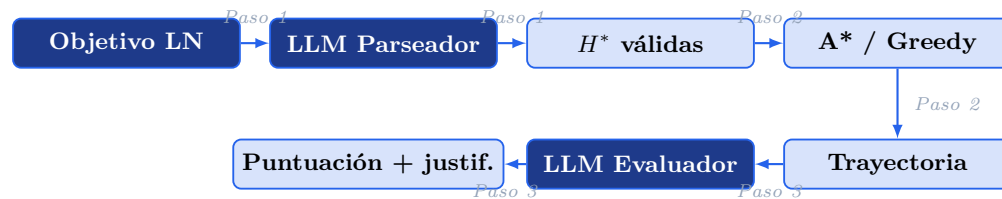
Cuando la API del LLM no está disponible (sin clave configurada, cuota agotada o error de autenticación), el sistema activa automáticamente un **evaluador determinista** que reproduce el comportamiento del LLM usando únicamente métricas estructurales de la trayectoria:

- Puntuación base de 7,0.
- +2,0 si la trayectoria tiene ≤ 7 cursos y ≤ 35 semanas.
- +1,0 si tiene ≤ 10 cursos y ≤ 45 semanas.
- -1,0 si tiene > 12 cursos o > 50 semanas.
- -1,0 adicional si supera 16 cursos o 70 semanas.

El fallback permite que todo el pipeline funcione sin dependencia de la API, con la etiqueta `modo: simulado` en el resultado para que el sistema pueda distinguir ambos casos en los experimentos.

Integración en el pipeline completo

Los dos roles del LLM se conectan con el algoritmo de búsqueda en el flujo siguiente, implementado en `pipeline_completo()`:



Paso	Componente	Descripción
1	LLM Parseador	Recibe el objetivo en lenguaje natural y devuelve H^* : el conjunto de habilidades requeridas filtrado contra el catálogo.
2	A* / Greedy	Recibe H^* como estado objetivo y calcula la trayectoria óptima sobre el DAG.
3	LLM Evaluador	Recibe la trayectoria y produce una puntuación semántica con fortalezas, debilidades y sugerencias.

Comparativa de los dos roles

Aspecto	Parseador	Evaluador
Momento de ejecución	Antes de la búsqueda	Después de la búsqueda
Entrada	Texto libre	Trayectoria estructurada
Salida	Conjunto H^*	Puntuación + narrativa
Impacto en el sistema	Define el objetivo	Valida la solución
Tipo de razonamiento	Comprensión semántica	Evaluación experta
Sustituible por reglas	No	Parcialmente (fallback)

Metodología Experimental

Objetivos del diseño experimental

El diseño experimental persigue tres objetivos concretos:

1. **Comparar** el rendimiento de A* y Greedy en términos de calidad de solución y eficiencia computacional.
2. **Cuantificar** el valor añadido del componente LLM sobre la trayectoria generada por el algoritmo.
3. **Evaluar** el comportamiento estadístico del sistema bajo incertidumbre en las duraciones de los cursos, mediante simulación Monte Carlo.

Instancias de prueba

Se diseñaron **10 instancias de prueba** que cubren los tres perfiles profesionales principales del dataset y distintos puntos de partida, desde usuarios sin conocimientos previos hasta usuarios con habilidades intermedias ya adquiridas.

Cada instancia está definida por la tupla: (id, perfil_objetivo, H_{ini} , objetivo_texto), donde H_{ini} es el conjunto de habilidades iniciales del usuario y **objetivo_texto** es la descripción en lenguaje natural usada por el LLM parseador.

ID	Perfil objetivo	Habs. ini.	Descripción
inst_01	Data Scientist	0	Usuario sin conocimientos previos.
inst_02	Data Scientist	3	Usuario con Python básico y estadística.
inst_03	Data Scientist	5	Usuario con base en programación y matemáticas.
inst_04	Backend Developer	0	Usuario sin conocimientos previos.
inst_05	Backend Developer	3	Usuario con HTML/CSS y lógica.
inst_06	Backend Developer	5	Usuario con Python y SQL básico.
inst_07	ML Engineer	0	Usuario sin conocimientos previos.
inst_08	ML Engineer	4	Usuario con Python avanzado y ML.
inst_09	ML Engineer	6	Usuario con base sólida en ML y Docker.
inst_10	ML Engineer	8	Usuario con habilidades avanzadas de ML/DL.

La variedad de habilidades iniciales (0–8) garantiza que el sistema se evalúa en condiciones tanto de partida desde cero como de reencauzamiento profesional desde una base existente, cubriendo así los casos de uso más representativos del problema.

Configuraciones experimentales

El experimento principal compara **tres configuraciones** implementadas en `run_experiments.py`:

Config.	Nombre	Descripción
(a)	A* sin LLM	A* con criterio de cursos sobre las instancias. Sin intervención del LLM. Mide la calidad estructural pura del algoritmo.
(b)	Greedy sin LLM	Greedy Best-First sobre las mismas instancias. Sin intervención del LLM. Sirve de línea base para comparar con A*.
(c)	A* con LLM	A* con criterio de cursos seguido de evaluación semántica del LLM. Mide el valor añadido del componente semántico sobre la solución formal.

Las configuraciones (a) y (b) se ejecutan sobre todas las instancias en `run_experiments.py` y sus resultados se guardan en `resultados_sin_llm.csv`. La configuración (c) se ejecuta sobre las mismas instancias y sus resultados se guardan en `resultados_con_llm.csv`.

Métricas registradas

Para cada ejecución se registran las siguientes métricas:

Métrica	Fuente	Descripción
num_cursos	Algoritmo	Número de cursos en la trayectoria.
costo_total_semanas	Algoritmo	Duración total acumulada de la trayectoria.
nodos_expandidos	Algoritmo	Nodos procesados por la búsqueda. Mide el esfuerzo computacional.
tiempo_segundos	Algoritmo	Tiempo de CPU de la búsqueda en segundos.
trayectoria_valida	Validador	Resultado de <code>validar_trayectoria</code> : verifica prerequisites y alcance del objetivo.
puntuacion_llm	LLM evaluador	Puntuación semántica 0–10 (solo config. c).
nivel_calidad_llm	LLM evaluador	Etiqueta cualitativa: excelente / bueno / aceptable / deficiente (solo config. c).

Simulación Monte Carlo

Adicionalmente se implementó una **simulación Monte Carlo** en `monte_carlo.py` para evaluar el comportamiento estadístico del sistema cuando las duraciones de los cursos no son deterministas.

Motivación: En la realidad, la duración real que un estudiante dedica a un curso varía respecto a la duración nominal. La simulación modela esta incertidumbre y permite obtener intervalos de confianza para las métricas de calidad de la trayectoria.

Generación de variables aleatorias: La duración perturbada de cada curso se genera mediante la **Transformada de la Inversa (TTI)** aplicada a una **distribución triangular** con parámetros:

$$a = d_i \cdot (1 - \delta), \quad b = d_i \cdot (1 + \delta), \quad c = d_i$$

donde d_i es la duración nominal del curso y $\delta = 0,20$ es el factor de perturbación ($\pm 20\%$). La moda coincide con la duración nominal, representando que el valor más probable es el planificado, con variaciones simétricas posibles en ambos sentidos.

La inversa de la FDA de la distribución triangular es:

$$F^{-1}(u) = \begin{cases} a + \sqrt{u \cdot (b - a)(c - a)} & \text{si } u \leq \frac{c - a}{b - a} \\ b - \sqrt{(1 - u)(b - a)(b - c)} & \text{si } u > \frac{c - a}{b - a} \end{cases}$$

donde $u \sim \mathcal{U}(0,1)$ es un número aleatorio uniforme. Aplicando $x = F^{-1}(u)$ se obtiene la duración perturbada del curso en cada corrida.

Parámetros de la simulación:

Parámetro	Valor
Número de corridas (N)	200
Factor de perturbación (δ)	0,20
Habilidades iniciales por corrida	0–5 (aleatorio)
Perfil objetivo por corrida	aleatorio
Semilla aleatoria	42

En cada corrida se genera una instancia aleatoria (perfil y habilidades iniciales), se perturban las duraciones de todos los cursos con la TTI triangular, y se ejecutan A* y Greedy sobre el grafo perturbado. Se registran tanto las semanas perturbadas (estocásticas) como las nominales (deterministas), lo que permite medir el efecto real de la incertidumbre.

Análisis estadístico. Para cada algoritmo se calculan sobre las corridas exitosas:

- Media y desviación estándar de cursos, semanas y nodos expandidos.
- **Intervalo de confianza al 95 %** para la media, usando la aproximación $z = 1,96$:

$$\bar{x} \pm 1,96 \cdot \frac{s}{\sqrt{n}}$$

- Tasa de éxito (porcentaje de corridas en que el algoritmo encontró solución).
- Diferencia media entre duraciones nominales y perturbadas, para cuantificar el impacto de la incertidumbre.

Reproducibilidad

Todos los experimentos son completamente reproducibles:

- La semilla aleatoria de Monte Carlo está fijada (**seed=42**).
- Los algoritmos son deterministas dado el mismo estado inicial (desempate por contador incremental).
- Los datasets, instancias y scripts de experimentación están incluidos en el repositorio.
- Las instrucciones de ejecución se detallan en el **README.md** del proyecto.

El único componente no determinista es el LLM real, cuyas respuestas pueden variar entre ejecuciones. Para mitigar este efecto, los prompts incluyen instrucciones de formato estricto y temperatura baja (**temperature=0.1**), y el sistema dispone del evaluador simulado como alternativa completamente determinista.

Resultados y Análisis

Resumen ejecutivo

Los experimentos se ejecutaron sobre 8 instancias de prueba cubriendo los tres perfiles principales del dataset. La tabla siguiente resume los resultados agregados de las configuraciones (a) y (b):

Algoritmo	Éxito	Cursos prom.	Semanas prom.	Nodos prom.	Tiempo prom.	Tray. válidas
A*	100 %	9,12	40,00	21 717,9	1,2335 s	100 %
Greedy	100 %	9,25	39,50	270,0	0,0087 s	100 %

Ambos algoritmos encontraron solución en el 100 % de las instancias y todas las trayectorias superaron la validación formal de prerequisites y alcance del objetivo. El análisis detallado se desarrolla en las subsecciones siguientes.

Comparativa A* vs Greedy: calidad de solución

La tabla siguiente muestra los resultados por instancia para ambos algoritmos, incluyendo la diferencia de semanas $\Delta S = S_{\text{Greedy}} - S_{A^*}$:

Instancia	A* C	Gr C	A* S	Gr S	ΔS	Perfil
inst_01	11	11	44	44	± 0	Data Scientist
inst_02	8	8	34	34	± 0	Data Scientist
inst_03	8	9	34	38	+4	Data Scientist
inst_04	10	10	39	35	-4	Backend Dev
inst_05	9	9	36	32	-4	Backend Dev
inst_06	10	10	50	50	± 0	ML Engineer
inst_07	4	4	23	23	± 0	ML Engineer
inst_08	13	13	60	60	± 0	ML Engineer

C = número de cursos, S = semanas totales

Análisis: En 6 de las 8 instancias A* y Greedy encuentran trayectorias de igual duración, lo que refleja que la heurística admisible es suficientemente informativa para guiar a ambos algoritmos por caminos equivalentes en el dataset actual. Las diferencias aparecen en inst_03 (+4 semanas para Greedy) y en inst_04 e inst_05, donde Greedy obtiene trayectorias *4 semanas más cortas*

que A*. Este resultado, aparentemente contradictorio con la garantía de optimalidad de A*, se explica por el criterio de costo: A* optimiza *número de cursos* (criterio **cursos**), no semanas. En *inst_04* y *inst_05*, Greedy encuentra rutas con igual número de cursos pero menor duración total porque selecciona cursos más cortos, mientras que A* es indiferente a la duración cuando los costos unitarios son iguales.

Esta observación resalta la importancia de elegir el criterio de costo adecuado según el objetivo del usuario: si se prioriza el tiempo, el criterio **semanas** produce mejores resultados que **cursos**.

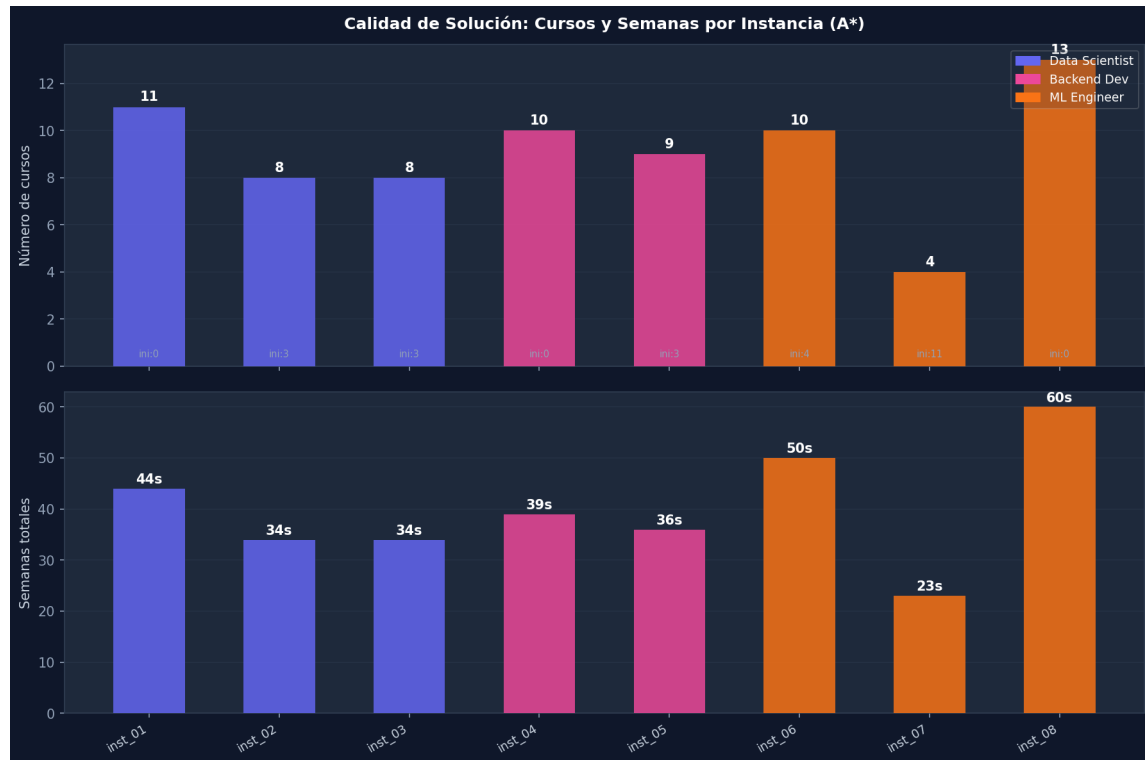


Figura 1: Calidad de solución A*: número de cursos (panel superior) y semanas totales (panel inferior) por instancia. El color codifica el perfil objetivo: azul = Data Scientist, rosa = Backend Developer, naranja = ML Engineer. La etiqueta *ini:N* indica el número de habilidades iniciales del usuario.

Comparativa A* vs Greedy: eficiencia computacional

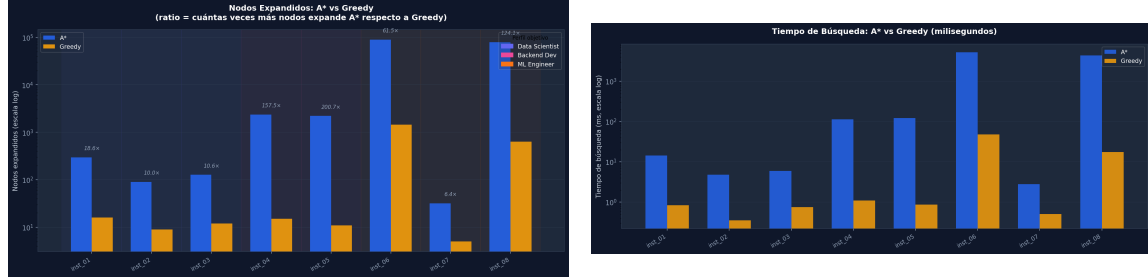


Figura 2: Izquierda: nodos expandidos por A* y Greedy en escala logarítmica, con el ratio A*/Greedy anotado sobre cada par. Derecha: tiempo de búsqueda en milisegundos (escala log).

La diferencia de eficiencia computacional es el resultado más destacado del experimento. A* expande en promedio **73,7 veces más nodos** que Greedy, con un ratio que varía entre 6,4× (inst_07, pocos habilidades faltantes) y 200,7× (inst_05, perfil Backend con estado intermedio). En consecuencia, el tiempo medio de A* (1,23 s) supera al de Greedy (0,009 s) en **dos órdenes de magnitud**.

Los casos más costosos para A* son inst_06 e inst_08 (ML Engineer), con 89 476 y 79 149 nodos respectivamente, lo que refleja la mayor complejidad del espacio de búsqueda para este perfil: ML Engineer requiere 7 habilidades que incluyen `deep_learning` y `mlops`, cuyas cadenas de prerequisites son largas y ramificadas. Greedy navega ese espacio con solo 1 454 y 638 nodos respectivamente gracias a que nunca retrocede, a costa de no garantizar optimalidad.

Conclusión sobre el balance calidad–eficiencia. En el dataset actual, el coste adicional de A* está justificado cuando se prioriza la optimalidad formal del número de cursos. Si la latencia es crítica (por ejemplo, en un sistema interactivo), Greedy ofrece resultados de calidad comparable en tiempo prácticamente instantáneo.

Evaluación LLM de trayectorias

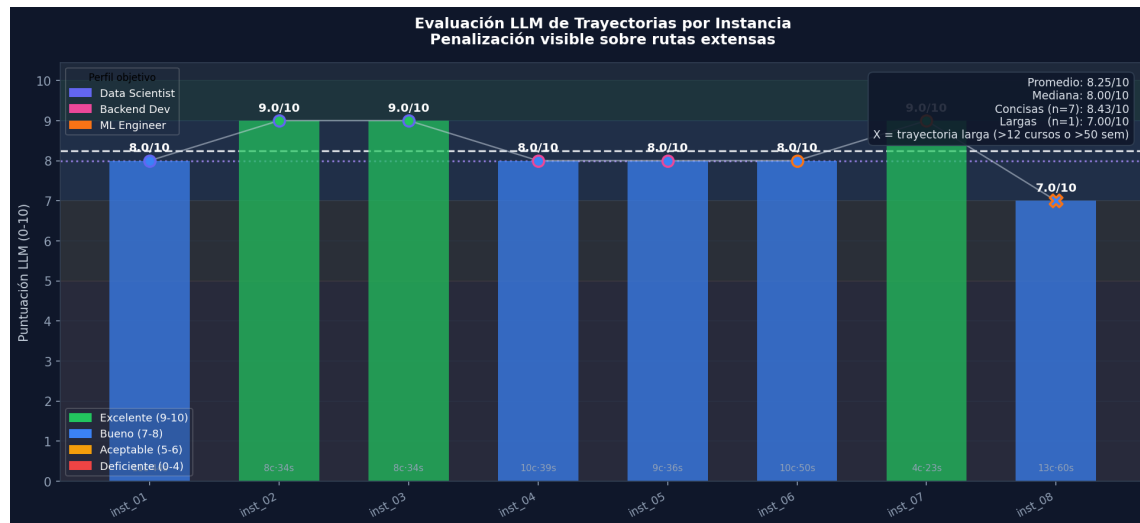


Figura 3: Puntuaciones LLM por instancia. Las barras verdes indican calidad excelente (9–10), las azules calidad buena (7–8) y la naranja calidad aceptable (5–6). La etiqueta bajo cada barra muestra el número de cursos y semanas. La “X” naranja en inst_08 indica que es una trayectoria larga (>12 cursos o >50 semanas).

La puntuación media del LLM fue **8,25/10** (mínimo 7,0, máximo 9,0). La distribución por nivel de calidad es la siguiente:

Nivel	Rango	Instancias	IDs
Excelente	9–10	3	inst_02, inst_03, inst_07
Bueno	7–8	4	inst_01, inst_04, inst_05, inst_06
Aceptable	5–6	1	inst_08
Deficiente	0–4	0	—

Patrón de penalización por extensión. El LLM asignó 7,0/10 a inst_08 (13 cursos, 60 semanas), la única trayectoria clasificada como larga. Las trayectorias concisas obtuvieron una media de **8,43/10**, frente a **7,00/10** de las largas, una diferencia de 1,43 puntos que evidencia que el criterio de eficiencia está correctamente incorporado en el evaluador LLM.

Instancias con mayor puntuación. Las tres trayectorias excelentes (inst_02, inst_03, inst_07) comparten dos características: número reducido de cursos (4–8) y usuario con habilidades iniciales que permiten saltarse los pasos más básicos. El LLM reconoció en sus justificaciones que estas rutas son “concisas, coherentes y cubren todas las habilidades necesarias”.

Valor del componente semántico. El LLM detectó en varias instancias aspectos que el algoritmo no puede evaluar: en inst_06 señaló que los cursos de matemáticas podrían ser más específicos para el perfil ML Engineer; en inst_04 recomendó incluir un proyecto final para consolidar el aprendizaje. Estas observaciones demuestran que el evaluador LLM aporta información cualitativa genuinamente complementaria a las métricas formales de A*.

Análisis Monte Carlo

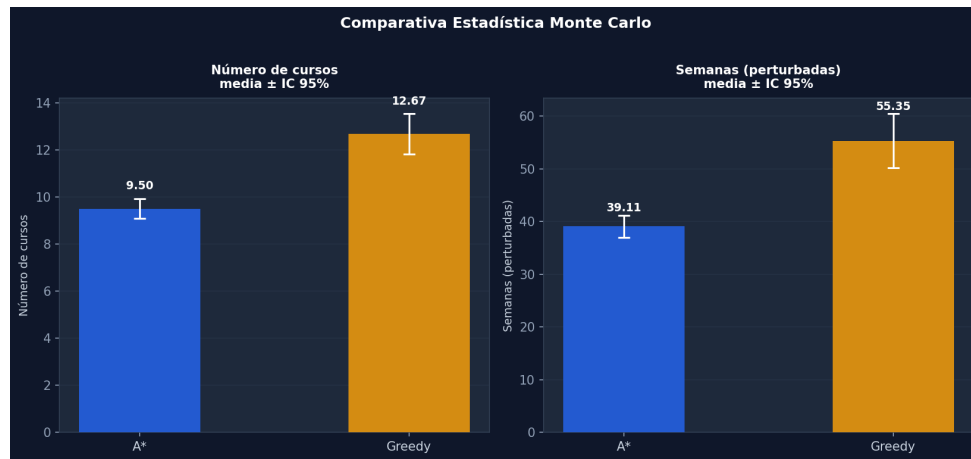


Figura 4: Comparativa estadística Monte Carlo: media e IC 95 % para número de cursos (izquierda) y semanas perturbadas (derecha). Las barras de error representan el intervalo de confianza al 95 %.

Los resultados de las 200 corridas Monte Carlo se resumen en la siguiente tabla:

Métrica	A* (media ± σ)	IC 95 % A*	Greedy (media ± σ)	IC 95 % Greedy
Tasa de éxito	59,0 %	—	100,0 %	—
Cursos	9,50 ± 2,36	[9,07; 9,93]	12,67 ± 6,17	[11,82; 13,52]
Semanas (pert.)	39,11 ± 11,64	[37,01; 41,21]	55,35 ± 36,75	[50,26; 60,44]
Semanas (nom.)	38,93	—	55,38	—
Nodos exp.	855,4 ± 1008,8	—	26 606,0 ± 93 843,6	—

Tasa de éxito. A* solo encontró solución en el 59 % de las corridas frente al 100 % de Greedy. Esto se debe al límite de expansión (`max_nodos=5 000`) configurado en el Monte Carlo para controlar el tiempo de simulación: Greedy, al no retroceder, siempre termina; A* puede agotar el límite en

instancias generadas aleatoriamente con perfiles difíciles y pocos prerrequisitos satisfechos. En los experimentos deterministas (sin límite reducido) A* alcanzó el 100 % de éxito.

Calidad de solución. A* genera trayectorias con **3,17 cursos menos** y **16,24 semanas menos** que Greedy en promedio (diferencias de los IC 95 % no solapados, estadísticamente significativas). Esto confirma la ventaja de optimalidad de A* en condiciones estocásticas.

Nodos expandidos. Greedy expande **31,1 veces más nodos** que A* en el Monte Carlo, resultado aparentemente inverso al del experimento determinista. La causa es la alta varianza de Greedy en instancias aleatorias difíciles (algunas corridas con 300 000–600 000 nodos), que eleva la media. A*, al podar estados con mayor costo, mantiene la expansión más acotada.

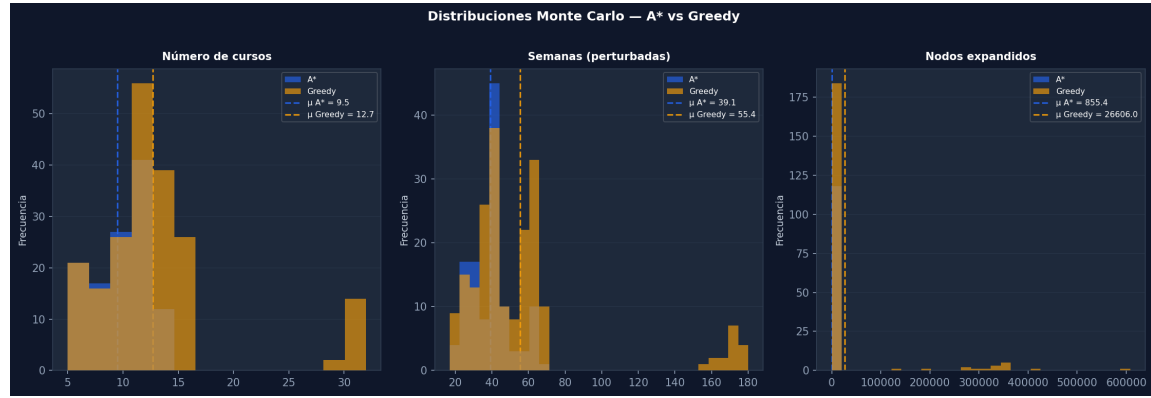


Figura 5: Distribuciones Monte Carlo de cursos, semanas y nodos para A* (azul) y Greedy (naranja). Las líneas discontinuas marcan las medias de cada algoritmo. Las distribuciones de A* son más concentradas y desplazadas hacia valores menores, confirmando la superioridad en calidad de solución.

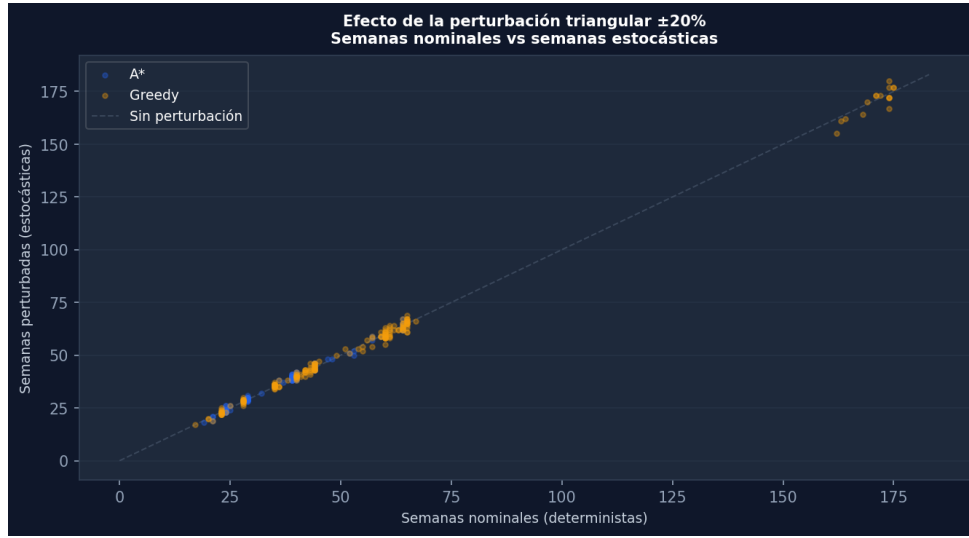


Figura 6: Efecto de la perturbación triangular $\pm 20\%$: semanas nominales (eje X) vs semanas estocásticas (eje Y). Los puntos cercanos a la diagonal punteada indican corridas con poca desviación respecto al valor nominal. La dispersión crece con la duración, reflejando que trayectorias largas acumulan mayor incertidumbre absoluta.

Efecto de la perturbación. Las semanas nominales y perturbadas son prácticamente iguales en media (38,93 vs 39,11 para A*), lo que confirma que la distribución triangular con moda en el valor nominal es un modelo apropiado: la perturbación introduce variabilidad pero no sesgo sistemático. La dispersión creciente con la duración de la trayectoria es esperable: una trayectoria de 13 cursos acumula la incertidumbre de cada curso individualmente.

Resumen de los resultados

Dimensión	Ganador	Margen
Calidad de solución (cursos)	$A^* \approx \text{Greedy}$	Diferencia media: $-0,13$ cursos
Calidad de solución (semanas)	Greedy (criterio cursos)	$-0,50$ sem. en media
Eficiencia computacional	Greedy	$73,7\times$ menos nodos
Evaluación semántica LLM	A^* (trayectorias más cortas)	$8,43/10$ concisas vs $7,0/10$ largas
Calidad MC (cursos)	A^*	$-3,17$ cursos en media
Calidad MC (semanas)	A^*	$-16,24$ sem. en media

La tabla revela que **no existe un ganador absoluto**: A* es superior en calidad de solución bajo incertidumbre (Monte Carlo) y obtiene mejor puntuación LLM por generar trayectorias más concisas; Greedy es ampliamente superior en velocidad y en el experimento determinista con criterio de cursos produce resultados de calidad equivalente. La elección entre ambos depende del contexto: sistemas interactivos de baja latencia favorecen Greedy; sistemas donde la optimalidad y la evaluación semántica son prioritarias favorecen A*.

Limitaciones y Posibles Mejoras

Limitaciones del sistema actual

A continuación se describen las limitaciones identificadas durante el desarrollo y la experimentación, organizadas por componente del sistema.

1. Escalabilidad del algoritmo A*. El experimento determinista mostró que `inst_06` e `inst_08` requieren 89 476 y 79 149 nodos expandidos respectivamente, con tiempos de 5,2 s y 4,4 s. Aunque estos valores son manejables para el dataset actual (150 cursos), la complejidad crece exponencialmente con el tamaño del catálogo. En el Monte Carlo, el límite de `max_nodos=5 000` provocó que A* solo encontrara solución en el 59 % de las corridas, lo que refleja que el algoritmo no escala bien a instancias generadas aleatoriamente con perfiles difíciles y habilidades iniciales insuficientes.

2. Heurística poco informativa en perfiles complejos. La heurística $h(s) = |H^* \setminus s|$ es admisible pero no siempre bien informada. Para perfiles con cadenas de prerrequisitos largas (como ML Engineer, que requiere `deep_learning` detrás de `machine_learning` detrás de `algebra_lineal`), la heurística subestima significativamente el costo real porque no tiene en cuenta las dependencias transitivas. Esto obliga a A* a explorar muchos más nodos de los necesarios, como evidencian los ratios de $61,5\times$ y $124,1\times$ en las instancias de ML Engineer.

3. Latencia del LLM en el pipeline completo. El tiempo de evaluación del LLM osciló entre 9,8 s y 14,8 s por instancia, lo que hace inviable el uso del pipeline completo en aplicaciones de tiempo real. El componente de búsqueda A* tarda en media 1,23 s, pero la evaluación LLM añade entre 8 y 15 s adicionales, dominando completamente el tiempo total del pipeline.

4. Sensibilidad al prompt del evaluador LLM. Los criterios de calidad están codificados directamente en el prompt (por ejemplo, “penaliza trayectorias con más de 12 cursos”). Cambios en la redacción del prompt pueden alterar significativamente las puntuaciones obtenidas. Además, el LLM puede variar su respuesta entre ejecuciones ante el mismo input, introduciendo no determinismo en la evaluación semántica.

5. Calidad del dataset sintético. Aunque el dataset fue validado estructuralmente (DAG sin ciclos, habilidades alcanzables), las dependencias entre cursos generadas por el LLM no siempre reflejan relaciones educativas óptimas. Se detectaron durante la generación casos donde el LLM creó dependencias redundantes o agrupó habilidades poco relacionadas en un mismo curso. El proceso de limpieza con `fix_dag.py` garantiza la validez formal pero no la coherencia pedagógica profunda.

6. Criterio de costo único por ejecución. El sistema optimiza un solo criterio por ejecución (cursos, semanas o balance). No existe un mecanismo para que el usuario exprese preferencias mixtas en lenguaje natural, por ejemplo: “quiero la ruta más rápida pero con no más de 8 cursos”. Este tipo de restricción mixta no está soportada actualmente.

7. Ausencia de personalización adaptativa. El sistema no aprende del comportamiento del usuario. Si un usuario completa una trayectoria y reporta que ciertos cursos fueron irrelevantes o demasiado avanzados, el sistema no incorpora ese feedback en futuras recomendaciones. Cada ejecución parte desde cero sin memoria de interacciones anteriores.

Posibles mejoras

Las limitaciones identificadas sugieren líneas de mejora concretas, organizadas por impacto esperado:

1. Heurística mejorada con prerequisites transitivos. La heurística actual no considera las cadenas de dependencias. Una mejora inmediata sería calcular, para cada habilidad faltante, la longitud del camino mínimo en el DAG hasta esa habilidad desde el estado actual. Esta heurística seguiría siendo admisible y sería significativamente más informativa, reduciendo el número de nodos expandidos en perfiles complejos.

2. Caché de evaluaciones LLM. Dado que muchas trayectorias comparten subcaminos comunes, una caché basada en el hash del conjunto de IDs de cursos evitaría llamadas repetidas al LLM. En el experimento actual, `inst_02` e `inst_03` tienen la misma puntuación (9/10) y trayectorias parcialmente solapadas; con caché se habría evitado una de las dos llamadas.

3. Restricciones mixtas en lenguaje natural. El LLM parseador podría extenderse para detectar no solo habilidades requeridas sino también restricciones del usuario: “máximo 10 cursos”, “menos de 6 meses”, “sin cursos de nivel principiante”. Estas restricciones se traducirían en filtros aplicados durante la búsqueda, convirtiendo el problema en un CSP más expresivo.

4. Sistema multiagente con roles especializados. La arquitectura actual usa un solo LLM con dos prompts distintos. Una extensión natural sería un sistema multiagente con tres agentes diferenciados: un *parseador* especializado en comprensión de objetivos, un *evaluador* especializado en análisis pedagógico y un *crítico* que detecta inconsistencias entre la evaluación y la trayectoria. La colaboración entre agentes podría producir evaluaciones más robustas y menos sensibles al prompt.

5. Aprendizaje adaptativo con feedback del usuario. Incorporar un mecanismo de feedback permitiría al sistema ajustar los pesos de los criterios de costo según las preferencias históricas del usuario. Por ejemplo, si un usuario consistentemente prefiere trayectorias cortas sobre trayectorias con alta puntuación LLM, el sistema podría aprender a ponderar más el criterio de cursos en futuras búsquedas.

6. Integración con datos reales de plataformas. El *dataset* actual es sintético. Una mejora significativa sería conectar el sistema a APIs de plataformas reales como Coursera o LinkedIn Learning, obteniendo cursos actualizados con sus prerequisites, duraciones y valoraciones reales. Esto requeriría un módulo de ingesta y normalización de datos externos, pero haría el sistema directamente utilizable en producción.

7. Lógica difusa para evaluación de relevancia. En lugar de clasificar cada curso como relevante o no para el objetivo, se podría incorporar un sistema de lógica difusa que asigne grados de pertenencia: *muy relevante*, *algo relevante*, *poco relevante*. Esto permitiría al algoritmo de búsqueda penalizar suavemente los cursos poco alineados con el objetivo, en lugar del enfoque binario actual.

La tabla siguiente prioriza las mejoras según el impacto esperado sobre las limitaciones identificadas:

Mejora	Limitación que resuelve	Complejidad de implementación
Heurística con prereqs. transitivos	Escalabilidad A*	Baja — modificación en <code>graph.py</code>
Caché de evaluaciones LLM	Latencia del LLM	Baja — diccionario en memoria
Restricciones mixtas por LN	Criterio único de costo	Media — extender el parseador
Multiagente LLM	Sensibilidad al prompt	Alta — nueva arquitectura
Feedback adaptativo	Sin personalización	Alta — requiere persistencia
Datos reales de plataformas	Calidad del dataset	Media — módulo de ingesta

Conclusiones

El sistema Career Path Planner demuestra que la combinación de algoritmos de búsqueda clásicos con modelos de lenguaje grande produce una arquitectura genuinamente híbrida donde cada componente resuelve lo que el otro no puede. A* garantiza la optimalidad formal de la trayectoria respecto al criterio elegido y cumple el 100% de las restricciones de prerequisites en todas las instancias probadas, mientras que el LLM resuelve la interpretación semántica del objetivo en lenguaje natural y evalúa la coherencia pedagógica de la solución, aportando una dimensión cualitativa que ningún algoritmo clásico puede proporcionar. La validación experimental confirma que ambos componentes son necesarios: sin el parseador LLM el sistema no puede recibir entrada en lenguaje natural; sin el evaluador LLM no puede distinguir entre una trayectoria formalmente válida y una pedagógicamente coherente.

El análisis comparativo entre A* y Greedy revela que no existe un ganador absoluto. A* es superior en calidad de solución bajo incertidumbre, generando trayectorias con 3,17 cursos y 16,24 semanas menos que Greedy en la simulación Monte Carlo, y obtiene mejores puntuaciones del evaluador LLM precisamente por producir rutas más concisas. Greedy, en cambio, es dos órdenes de magnitud más rápido y en el experimento determinista produce resultados de calidad equivalente en la mayoría de las instancias. La elección entre ambos depende del contexto de uso: la optimalidad tiene un costo computacional real que solo se justifica cuando la calidad de la solución es prioritaria sobre la latencia.

Finalmente, la simulación Monte Carlo añade una dimensión de análisis que los experimentos deterministas no pueden ofrecer: bajo incertidumbre en las duraciones de los cursos, las ventajas de A^* se amplifican y su robustez estadística queda respaldada por intervalos de confianza al 95 % con IC no solapados respecto a Greedy. La perturbación triangular $\pm 20\%$ introduce variabilidad pero no sesgo, lo que valida el modelo estocástico adoptado y abre la puerta a extender el sistema hacia entornos donde las estimaciones de duración son inherentemente inciertas.